

Un linguaggio per domarli tutti: applicazioni full stack con librerie native in Kotlin Multiplatform

Sohaib Ouakani

Corso di Laurea in Ingegneria e Scienze informatiche

Relatore: Prof. Danilo Pianini

Correlatori: Dott. Angela Cortecchia, Ing. Valter Venusti

A.A. 2025/2026

- 1 Contesto
- 2 Approcci esistenti
- 3 La Soluzione Proposta
- 4 Standard FMI e formato FMU (Background)
- 5 Architettura
- 6 Architettura: Dettaglio
- 7 Supporto sperimentale a sistemi Apple Silicon
- 8 Valutazione
- 9 Conclusioni e sviluppi futuri

Contesto: stack software eterogeneo

- Stack industriali spesso eterogenei
- Librerie di standard tecnici scritte in linguaggi nativi
- Risultato: duplicazione di codice, complessità, manutenzione difficile

Problema

Si può usare un solo linguaggio per l'intero applicativo, mantenendo l'accesso a librerie native di dominio?

- Node.js: molto popolare, ma interazione con il nativo complessa
- Linguaggi fullstack popolari richiedono spesso codice in altri linguaggi

Limite principale

Serve codice legante in C per interagire con librerie native

Kotlin Multiplatform + CInterop

Applicazione fullstack multiplatforma in Kotlin, con accesso diretto al nativo tramite CInterop

- Un solo linguaggio per tutto lo stack: Kotlin
- Interfaccia diretta con C tramite CInterop
- Nessun layer di astrazione intermedio

- FMI: standard per lo scambio di modelli di simulazione, con interfaccia C
- FMU: archivio zip con modello e metadati, secondo lo standard FMI

Perché sono rilevanti

Caso di studio per validare l'approccio Kotlin fullstack in ambito industriale

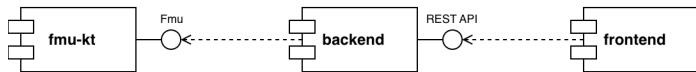


Figura: Architettura generale del sistema.

Linguaggi

- Frontend: **Kotlin/JS**
- Backend: **Kotlin/Native**
- Logica di dominio (fmu-kt): **Kotlin/Native**
- Stessa sintassi e semantica
- Librerie, backend e runtime diversi

Dettaglio dell'architettura

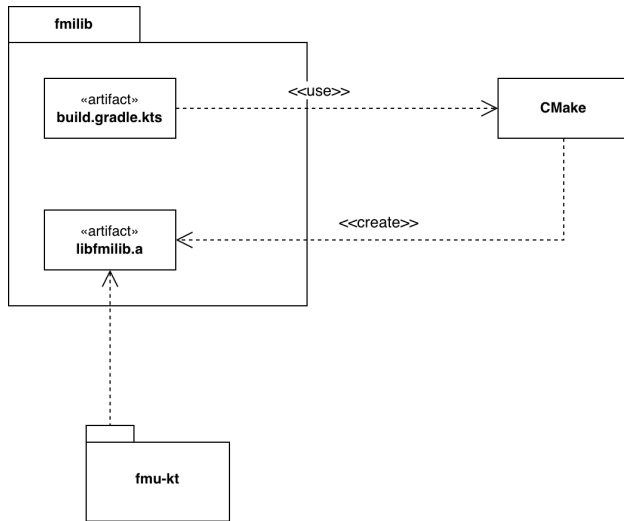
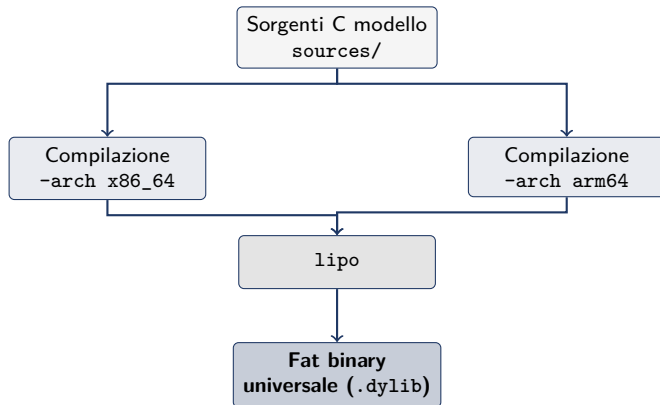


Figura: Come avviene l'integrazione con il codice nativo.

Supporto sperimentale a sistemi Apple Silicon

- FMU spesso solo in x86_64, incompatibili con Apple Silicon
- Rosetta 2 non utilizzabile (caricamento dinamico via dlopen)



- Test unitari dei moduli
- Test di integrazione su FMU pubblici (submodule git + Kotest)
- CI su GitHub Actions: Windows, Linux, macOS

Sintesi

Sistema validato, non solo compilato, su tutte le piattaforme target

Obiettivi raggiunti

- Caricamento FMU (+ ricompilazione su macOS)
- Visualizzazione metadati e variabili
- Configurazione simulazione (variabili, durata, passo)
- Grafico dei risultati

Limitazioni

- Solo standard FMI 2.0
- Ricompilazione macOS: soluzione best-effort
- Kotlin/Native: stdlib limitata, più lavoro manuale (es. API POSIX)

Conclusioni

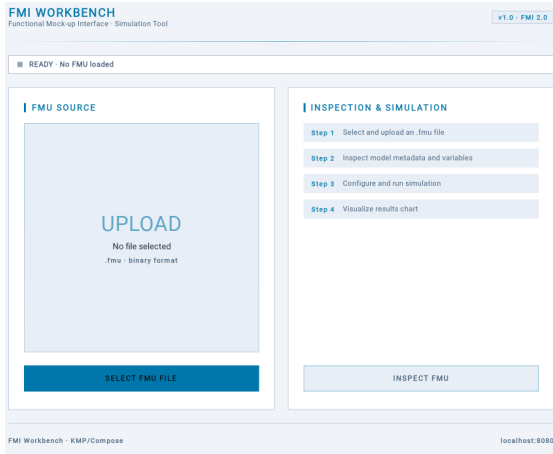


Figura: Schermata di upload.

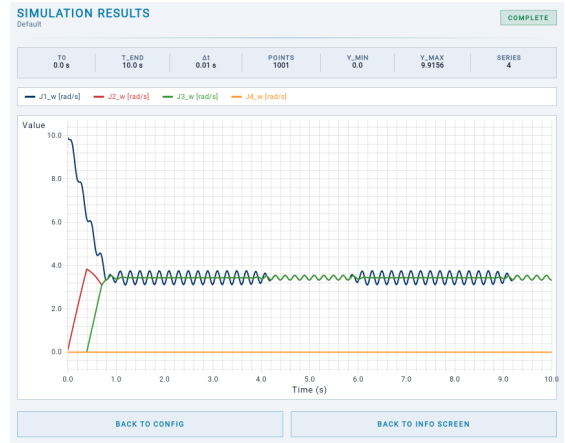


Figura: Grafico di una simulazione.

Sviluppi futuri

- Supporto ad altre versioni dello standard FMI
- Esportazione dei risultati della simulazione
- Gestione di più modelli FMU in contemporanea

Grazie per l'attenzione